

Module 3

(CFG).

Context Free Grammars:-

* It is a notation used to specify the syntax of language.
* CFG are used to design parser.

* Definition:- A context-free Grammar (CFG) consisting of a finite set of grammar rules is a quadruple (V, T, P, S) where
* V is a set of variables or non-terminal symbols.
* T is a set of terminals
* P is a set of productions of the form $A \rightarrow \alpha$
* S is the start symbol.

Ex:- $E \rightarrow E \cup T / T$.

$T \rightarrow T^* F / F$.

$F \rightarrow (E) / id$.

$T \rightarrow (E, 1) x$

Here, $V = \{E, T, F\}$.

$T = \{+, *, (,), id\}$.

$P = \{E \rightarrow E \cup F, E \rightarrow T, T \rightarrow T^* F, T \rightarrow F, F \rightarrow (E), F \rightarrow id\}$

$S = \{E\}$.

Ex:-

Write a CFG to recognize the language $L = \{a^n b^n \mid n \geq 0\}$ is

$S \rightarrow aSb \mid \epsilon$.

$L = \{ \epsilon, ab, aabb, \dots \}$.

CFG, $G = \{S, \{a, b\}, P, S\}$.

Ex:- $aabb$.

$S \rightarrow aSb$.

$\rightarrow aaaSb$.

$\rightarrow aabb$.

1) Obtain a grammar to generate a string consisting any no. of a 's on $\Sigma = \{a\}$.

$L = \{a^n \mid n \geq 0\}$

$L = \{\epsilon, a, aa, \dots\}$.

$S \rightarrow aS \mid \epsilon$.

CFG, $G = \{S, \{a\}, P, S\}$.

Ex:- aaa .

$S \rightarrow aS$

$\rightarrow aaaS$

$\rightarrow aaaa$

$\rightarrow aaaa$

$\rightarrow aaaa$

2) Obtain a grammar to generate a string consisting of

at least one a on $\Sigma = \{a\}$.

$L = \{a^n \mid n \geq 1\}$.

$L = \{a, aa, aaa, \dots\}$.

$S \rightarrow aS \mid a \rightarrow$ otherwise infinite loop.

$G = \{S, \{a\}, P, S\}$.

Ex:- $aaaa$.

$S \rightarrow aS$
 $\rightarrow aaS$
 $\rightarrow \underline{aaa}$

3) Obtain a grammar to generate a string consisting of any no. of a's and b's.

$S \rightarrow aS / bS / \epsilon$

$G = \{S, \{a, b\}, P, S\}$

Ex: $ababb$

$S \rightarrow aS$
 $\rightarrow abS$
 $\rightarrow abas$
 $\rightarrow ababS$
 $\rightarrow ababbs$
 $\rightarrow ababb\epsilon$
 $\rightarrow \underline{ababb}$

4) Obtain a grammar to generate at least 2 a's.

$S \rightarrow aaA$

$A \rightarrow aA / \epsilon$

$G = \{S, A, \{a\}, P, \{S\}\}$

Ex: $S \rightarrow aaaaaa$

$\rightarrow aaA$
 $\rightarrow aaapA$
 $\rightarrow aaaaaA$
 $\rightarrow \underline{aaaaaa}$

Module 3

Q) Multiple of 3 or 5.

$S \rightarrow aaaS | \epsilon$.

$G = \{S, \{a\}, P, \{S\}\}$.

Ex:- aaaS aaaS

$S \rightarrow aaaS$

$\rightarrow aaaS aaaS$

$\rightarrow aaaS aaaS$

$\Rightarrow$ String length is multiple of 3 or 5 $\Rightarrow \{a, b\}$

$S \rightarrow aAAS | \epsilon$.

$A \rightarrow a | b$.

$G = \{S, A, \{a, b\}, P, \{S\}\}$.

Ex:- ababab

$S \rightarrow AAS$

$\rightarrow aAAS$

$\rightarrow abAS$

$\rightarrow abAS$

$\rightarrow abAAS$

$\rightarrow ababAS$

$\rightarrow ababAAS$

$\rightarrow ababab$

Q) Any no. of a's & b's with at least one a.

Q) At least one a or one b.

Q) $L = \{w : |w| \bmod 3 > 0, \Sigma = \{a, b\}\}$.

$S \rightarrow a|aa|aaa$.

$G = \{S, \{a\}, P, \{S\}\}$.

Ex aaaa.

$S \rightarrow aaaS$

$\rightarrow aaaa$

Q) $S \rightarrow a^2 | a^3 | a^4 | \epsilon$

$G = \{S, \{a\}, P, \{S\}\}$.

Ex:- abaab.

$S \rightarrow aS \rightarrow abS \rightarrow abas \rightarrow abaaS \rightarrow abaaabS$

$\rightarrow abaaab$

Q) $S \rightarrow aS | bS | a | b$

$G = \{S, \{a, b\}, P, \{S\}\}$

Ex:- aaba

$S \rightarrow aS$

$\rightarrow aaaS$

$\rightarrow aabS$

$\rightarrow aabaS$

$\rightarrow$

11) Obtain a grammar to generate strings of a^5 and b^5 having subtring ab .

$S \rightarrow A ab A$
 $A \rightarrow a A | b A | \epsilon$

$G = \{ \{S, A\}, \{a, b\}, P, \{S\} \}$

Ex:- $bbabaa$

$S \rightarrow A ab A$
 $\rightarrow b A ab A$
 $\rightarrow \underline{bbabaa}$

12) Ending with string ab .

$S \rightarrow A ab$
 $A \rightarrow a A | b A | \epsilon$

$G = \{ \{S, A\}, \{a, b\}, P, \{S\} \}$

Ex:- bab
 $S \rightarrow A ab$
 $\rightarrow b A ab$
 $\rightarrow \underline{bab}$

13) $L = \{ w : N_a(w) \bmod 2 = 0 \text{ on } \Sigma = \{a, b\} \}$

(Ca-seven)

$S \rightarrow S a S a S | b S | \epsilon$

$G = \{ S, \{a, b\}, P, \{S\} \}$

Ex:- $babab$

$S \rightarrow S a S a S$
 $\rightarrow b S a S a S$
 $\rightarrow b a S a S \rightarrow b a b S a S \rightarrow b a b b S a S \rightarrow b a b b a S$
 $\rightarrow \underline{babab}$

14) $L = \{ a^n b^n, n \geq 0 \}$

$S \rightarrow a S b | \epsilon$

$G = \{ S, \{a, b\}, P, \{S\} \}$

Ex:- $aabbb$

$S \rightarrow a S b$
 $\rightarrow a a S b b$
 $\rightarrow \underline{a a b b b}$

15) $L = \{ a^n b^n, n \geq 1 \}$

$S \rightarrow a S b | ab$

$G = \{ S, \{a, b\}, P, \{S\} \}$

Ex:- ab $aabbb$
 $S \rightarrow a S b$
 $\rightarrow \underline{a a b b b}$

16) $L = \{ a^m b^n, n \geq 0 \}$

$S \rightarrow a S b | a$

$G = \{ S, \{a, b\}, P, \{S\} \}$

Ex:- aab

$S \rightarrow a S b$
 $\rightarrow \underline{a a b}$

17) $L = \{ a^n b^{n+1}, n \geq 0 \}$

$S \rightarrow a S b | b$

$G = \{ S, \{a, b\}, P, \{S\} \}$

Ex:- $aabbb$
 $S \rightarrow a S b$
 $\rightarrow a a S b b$
 $\rightarrow \underline{a a b b b}$

18) $L = \{a^n b^m, n \geq 0\}$

$S \rightarrow a S b / b \epsilon$

$G = \{S, \{a, b\}, P, \{S\}\}$

Ex:- $aabbbb$

$S \rightarrow a S b$

$\rightarrow a S b b$

$\rightarrow a a b b b b$

19) $L = \{a^n b^m, n \geq 0\}$

$S \rightarrow a S b b / a b b \epsilon$

$G = \{S, \{a, b\}, P, \{S\}\}$

Ex:- $a a b b b b$

$S \rightarrow a S b b$

$\rightarrow a a b b b b$

$\rightarrow a a$

20) $L = \{w w^R : w \in \{a, b\}^*\}$

$S \rightarrow a S a / b S b \epsilon$

$G = \{S, \{a, b\}, P, \{S\}\}$

Ex:- $a b b a$

$S \rightarrow a S b$

$\rightarrow a b S b a$

$\rightarrow a b b a$

21) $L = \{0^m 1^m 2^n : m \geq 1, n \geq 0\}$

$S \rightarrow A B$

$A \rightarrow 0 A 1 / 0 1$

$B \rightarrow 2 B / \epsilon$

$G = \{S, A, B, \{0, 1, 2\}, P, \{S\}\}$

Ex:- 0122

$S \rightarrow A B$

$\rightarrow 0 A 1 B$

$\rightarrow 0 1 B$

$\rightarrow 0 1 2 B$

$\rightarrow 0 1 2 2 B$

$\rightarrow 0 1 2 2$

22) What is the language generated by the grammar.

$S \rightarrow 0 A / \epsilon$

$A \rightarrow 1 S$

$L = \{ (01)^n, n \geq 0 \}$

Ex:-

$S \rightarrow 0 A$

$\rightarrow 0 1 S$

$\rightarrow 0 1 0 A$

$\rightarrow 0 1 0 1 S$

$\rightarrow 0 1 0 1$

23) Obtain a grammar to generate language, $L = a^n b^m / n \geq 0, m \geq n$

$m \geq n$

$n = 0$

$m \geq 1$

$\frac{a b b^*}{a b}$

$n = 1$

$m \geq 2$

$\frac{a b b^*}{a b}$

$m \geq 3$

$\frac{a a b b b^*}{a b}$

$a^n b^n : n \geq 0$

$\rightarrow$ one or more bs

$b^n : n \geq 1$

$S \rightarrow AB$

$A \rightarrow aAb|e$

$B \rightarrow bB|b$

$G = \{S, A, B\}, \{a, b\}, P, \{S\}$

Ex: $aabbb$

$S \rightarrow AB$

$\rightarrow aAbB$

$\rightarrow aaAbB$

$\rightarrow aabbbB$

$\rightarrow aabbbB \rightarrow aabbb$

20) $L = a^n b^{n-2} \mid n \geq 3$

$n=3$

$a^3 aab|e$

A

$n=4$

$aaabab$

A

$n=5$

$aaaababb$

A

$S \rightarrow AB$

$A \rightarrow aaa$

$B \rightarrow aBb|e$

$G = \{S, A, B\}, \{a, b\}, P, \{S\}$

Ex: $aaaab$

$S \rightarrow AB$

$\rightarrow aaAB$

$\rightarrow aaaABb$

$\rightarrow aaaaB$

25) $L = \{w \mid |w| \bmod 5 \neq |w| \bmod 2 \text{ on } \Sigma = \{a\}\}$

$0 \bmod 5 = 0$

$0 \bmod 2 = 0$

$1 \bmod 5 = 1$

$1 \bmod 2 = 1$

$2 \bmod 5 = 2$

$2 \bmod 2 = 0$

$S \rightarrow aa|aaa|aaaa|aaaaa|aaaaaa$

Ex: $aaaaaaa$

$S \rightarrow aaaaaa$

$\rightarrow aaaaaaa$

26) No more than 3 a's on $\{a, b\}$

$S \rightarrow B|BAB|BABAB|BABABAB$

$B \rightarrow bB|e$

$G = \{S, B\}, \{a, b\}, P, \{S\}$

Ex: $bababa$

$S \rightarrow BABAB$

$\rightarrow bBABAB$

$\rightarrow bbBABAB$

$\rightarrow bbbBABAB$

27) Grammar to generate a language of 0's & 1's having

Substring 000.

$S \rightarrow A000A$

$A \rightarrow 0A|1A|e$

$G = \{S, A\}, \{0, 1\}, P, \{S\}$

Ex: 11000

$S \rightarrow A000A$

$\rightarrow 1A000$

$\rightarrow 11A000$

$\rightarrow 11000$

28) Write a grammar to generate an arithmetic expression

Using operators $+$, $-$, $*$, $/$ & identifiers $\{a, b, c\}$.

$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid T$

$T \rightarrow a \mid b \mid c \mid (E) \mid \neg$

$G = \{ \{E, T\}, \{+, -, *, /\}, \{a, b, c\}, P, \{E\} \}$

Ex:- $a + b * c / b$

$E \rightarrow E + E$

$\rightarrow a + E$

$\rightarrow a + E * E$

$\rightarrow a + T * E \rightarrow a + b * E \rightarrow a + b * T \rightarrow a + b * c / b$

$\rightarrow a + b * c / b$

29) $L = \{ 0^i 1^j \mid i \neq j, i, j \geq 1 \}$
equal no's of 0's & 1's

$S \rightarrow 0S1 \mid A \mid B$ more 0's

$A \rightarrow 0A \mid 0$

$B \rightarrow bB \mid b$

$G = \{ \{S, A, B\}, \{0, 1\}, P, \{S\} \}$

Ex:- 000011

$S \rightarrow 0S1$

$\rightarrow 00S11$

$\rightarrow 000S111$

$\rightarrow 000011$

30) $L = \{ a^n b^m c^k \mid 2m+n=k, n, m \geq 0 \}$

$L = \{ a^n b^m c^{2m+n} \}$

$L = \{ a^n b^m c^{2m} c^n \}$
equal no's of c's

$S \rightarrow aSc \mid B \rightarrow b^m c^m$

$B \rightarrow bBc \mid bcc$

$G = \{ \{S, B\}, \{a, b, c\}, P, \{S\} \}$

Ex:- $0000cccccc$

$S \rightarrow aSc$

$\rightarrow 0000cc$

$\rightarrow 0000cc \rightarrow 0000cccc \rightarrow 0000cccccc$

$\rightarrow 0000cccccc$

31) $L = \{ w \mid n_a(w) = n_b(w) + 1 \}$

$S \rightarrow aAa$

$A \rightarrow aAb \mid bAa \mid A \mid \epsilon$

$G = \{ \{S, A\}, \{a, b\}, P, \{S\} \}$

Ex:- 00000000

$S \rightarrow AaA$

$\rightarrow aAa$

$\rightarrow aAa$

$\rightarrow aAaAa$

$\rightarrow 00000000$

$\rightarrow 00000000$

$\rightarrow 00000000$

00000000

$S \rightarrow AaA$

$\rightarrow Aa$

$\rightarrow AaA$

$\rightarrow AaAaA$

$\rightarrow 00000000$

$\rightarrow 00000000$

$\rightarrow 00000000$

38) $L = \{w \mid n_a(w) > n_b(w) \}$

$G \rightarrow AB \mid BA \mid BAB$ some a's & b's

$A \rightarrow aAb \mid bAa \mid AaA$ (for equal a & b)

$B \rightarrow aB \mid a$

$G = \{ \{s, a, b\}, \{a, b\}, p, \{s\} \}$

Ex: $aabab$

$S \rightarrow BA \rightarrow BAb \rightarrow Baab \rightarrow Baab$

$\rightarrow aabab$

39) $L = \{a^n b^m \mid n_a(w) > n_b(w) \}$ finite (23)

$m=0 \quad m=1 \quad m=2$

$n \geq 1 \quad n \geq 2 \quad n \geq 3$

$\frac{aa^* \epsilon}{a} \quad \frac{aa^* b}{a} \quad \frac{aaab}{a}$

$S \rightarrow AB \mid BA$

$B \rightarrow aB \mid a$ $G = \{ \{s, a, b\}, \{a, b\}, p, \{s\} \}$

Ex: $aaabab \rightarrow BAb \rightarrow Baab \rightarrow Baab \rightarrow aabab$

34) $L = \{w \mid n_a(w) = n_b(w) \}$

$S \rightarrow aSb \mid bSa \mid \epsilon$ $S \rightarrow aSb \mid bSa \mid S \mid \epsilon$

$G = \{ \{s, a, b\}, \{a, b\}, p, \{s\} \}$ $G = \{ \{s, a, b\}, p, \{s\} \}$

Ex: $abbaaabbba \quad abbaaabbba$

$S \rightarrow SS \rightarrow aSbS \rightarrow abS \rightarrow abSS \rightarrow abSaaS \rightarrow abbaS$

$\rightarrow abbaSS \rightarrow abbaaSbS \rightarrow abbaaSaSS \rightarrow abbaaSaSS$

$\rightarrow abbaaSaSS \rightarrow abbaaSaSS$

40) $L = \{a^n b^m c^m \mid n, m > 0\}$

$G \rightarrow AB$

$A \rightarrow aAb \mid \epsilon$

$B \rightarrow cB \mid \epsilon$

$G = \{ \{s, a, b, c\}, \{a, b, c\}, p, \{s\} \}$

Ex: $aabacc$

$S \rightarrow AB$

$\rightarrow aAbB$

$\rightarrow aAabB$

$\rightarrow aabB$

$\rightarrow aabBcB$

$\rightarrow aabBccB$

$\rightarrow aabBccc$

36) $L = \{w \mid \text{where } w \text{ is a string of delimiters: } (,), [,], \{, \}, \}$
that are properly balanced

$S \rightarrow (S) \mid [S] \mid \{S\} \mid \epsilon$

$G = \{ \{ \{ (,), [,], \{ , \} \}, \{ (,), [,], \{ , \} \}, p, \{ \{ (,), [,], \{ , \} \} \}$

Ex: $\{ (() [] \{ \} \}$

$S \rightarrow \{S\}$

$\rightarrow \{ \{S\} \}$

$\rightarrow \{ \{ \{S\} \} \}$

$\rightarrow \{ \{ (S) \{S\} \}$

$\rightarrow \{ (() [] \{ \} \}$

$\rightarrow \{ (() [] \{ \} \}$

$\rightarrow \{ (() [] \{ \} \}$

ex) $L = \{w \in \{a,b\}^* : \#a(w) = 2\#b(w)\}$.

$S \Rightarrow aSaaSbs \mid bSaaSas \mid SaaSbs \mid aS$

$G = \{S, \{a,b\}, \{ \}, \{S \Rightarrow aSaaSbs, S \Rightarrow bSaaSas, S \Rightarrow SaaSbs, S \Rightarrow aS\} \}$.

EX:- $aabab$

$S \Rightarrow aSaaSbs$

$\rightarrow aSaabSaaSbs$

$\rightarrow aSaaSbs$

$\rightarrow aaaSbs$

$\rightarrow aaaSbs$

$\rightarrow aaaSaaSbsbs$

$\rightarrow aaaSbs$

Derivation using a grammar:-

* The process of obtaining a string of terminals or non-terminals from the start symbol by applying production is called derivation. It is denoted by $\Rightarrow$.

* Types of derivation:-

1) Left Most Derivation (LMD)

2) Right Most Derivation (RMD)

* EX:- Consider the grammar $E \Rightarrow E+E \mid E^*E \mid id$. obtain a string id^*id+id .

$E \Rightarrow E+E$

$\Rightarrow E^*E+E$

$\Rightarrow id^*E+E \Rightarrow id^*id+E \Rightarrow id^*id+id$

* LMD:- In the derivation process if a left most variable is replaced at every step then the derivation is said to be left most derivation.

* RMD:- In the derivation process if a right most variable is replaced at every step then the derivation is said to RMD.

EX:-

1) Consider the Grammar $E \Rightarrow E+E \mid E^*E \mid id$. obtain

LMD and RMD for a string $id+id^*id$.

$E \Rightarrow E+E$

$\Rightarrow id+E$

$\Rightarrow id+E^*E \Rightarrow id+id^*E \Rightarrow id+id^*id$

$E \Rightarrow E^*E$

$\Rightarrow E^*id \Rightarrow E+E^*id \Rightarrow E+id^*id \Rightarrow id+id^*id$

* Left Sentential Form:- If there is a derivation of the form $S \Rightarrow_{lm} \alpha$ where at each step in the derivation process only a left most variable is replaced then ' α ' is called left Sentential Form.

EX:- $E \Rightarrow_{lm} E+E \Rightarrow_{lm} id+id+E$

* Right Sentential Form:- If there is a derivation of the form $S \Rightarrow_{rm} \alpha$ where at each step in the derivation process only a right most variable is replaced then ' α ' is called.

right sentential form.

$$E \xRightarrow{rm} E + E + E$$

$$\xRightarrow{rm} etid.$$

Parse Tree (Derivation Tree) :-

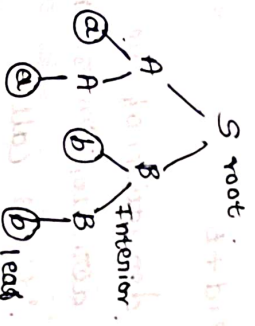
* Let $G = (V, T, P, S)$ be a CFG then the parse tree is defined by the following properties:

- * The root has label 'S'
- * Every vertex has a label which is in $(V \cup T)$
- * Every leaf node has label from 'T' & an interior vertex has a label from 'V'.

* If a vertex is labeled 'A' and if $x_1, x_2, \dots, x_n$ are the children of A then $A \rightarrow x_1, x_2, \dots, x_n$ must be a production in P.

Ex:- aabb.

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow aA/a \\ B &\rightarrow bB/b \end{aligned}$$



Obtain LMD and RMD and parse tree for $+^* - xyxy$ for the grammar $E \rightarrow +E \mid *E \mid -E \mid x \mid y$.

$$LMD:- E \xRightarrow{lm} +EE$$

$$\xRightarrow{lm} +^*EE$$

$$\xRightarrow{lm} +^* -EE$$

$$\xRightarrow{lm} +^* -xEEE$$

$$\xRightarrow{lm} +^* -xyxy$$

$$\xRightarrow{lm} +^* -xyxE \xRightarrow{lm} +^* -xyxy$$

$$RMD:- E \xRightarrow{rm} +EE$$

$$\xRightarrow{rm} +Ey$$

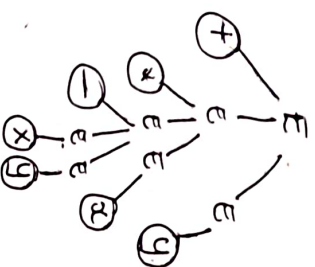
$$\xRightarrow{rm} +^*Eey$$

$$\xRightarrow{rm} +^*Eexy$$

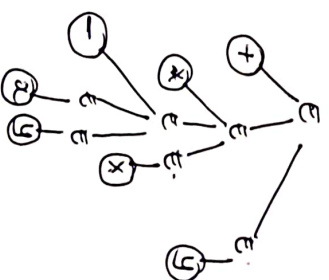
$$\xRightarrow{rm} +^* -Eexy$$

$$\xRightarrow{rm} +^* -Eyx \xRightarrow{rm} +^* -xyxy$$

parse tree:- LMD:-



RMD:-



Module 3

8) Obtain LMD, RMD and parse tree for the string $-(id+id)$ using $E \rightarrow E+E / E^*E / -E / (E) / id$.

LMD: $E \xRightarrow{lm} -E$

$$\xRightarrow{lm} -(E)$$

$$\xRightarrow{lm} -(E+E)$$

$$\xRightarrow{lm} -(id+E)$$

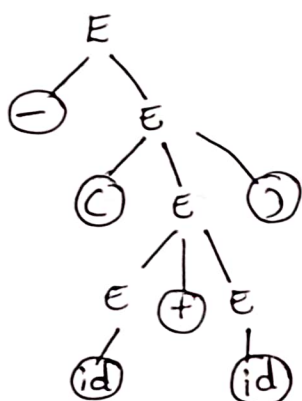
$$\xRightarrow{lm} \underline{-(id+id)}$$

RMD: $E \xRightarrow{rm} -E$

$$\xRightarrow{rm} -(E)$$

$$\xRightarrow{rm} -(E+E) \xRightarrow{rm} -(E+id) \xRightarrow{rm} \underline{-(id+id)}$$

Parse tree:-



9) Obtain LMD, RMD and parse tree for string $aa+a^*$ using $S \rightarrow SS+ / SS^* / a$.

LMD: $S \xRightarrow{lm} SS^*$

$$\xRightarrow{lm} SS+S^*$$

$$\xRightarrow{lm} aS+S^*$$

$$\xRightarrow{lm} aa+S^*$$

$$\xRightarrow{lm} \underline{aa+a^*}$$

RMD: $S \xRightarrow{rm} SS^*$

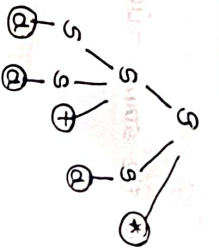
$$\xRightarrow{rm} SS+a^*$$

$$\xRightarrow{rm} aa+a^*$$

$$\xRightarrow{rm} \underline{aa+a^*}$$

$$\xRightarrow{rm} \underline{aa+a^*}$$

Parse tree:-



4) Obtain LMP, RMP and parse tree for string t^*aa using

$S \rightarrow tS \mid *SS \mid a$.

LMP:- $S \xRightarrow{lm} tS$

$\xRightarrow{lm} t^*SS$

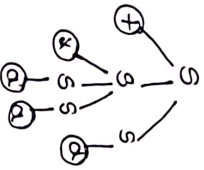
$\xRightarrow{lm} t^*aSS \xRightarrow{lm} t^*aaS \xRightarrow{rm} t^*aad$

RMP:- $S \xRightarrow{rm} tS$

$\xRightarrow{rm} t^*SSS \xRightarrow{rm} t^*SSa$

$\xRightarrow{rm} t^*SSa \xRightarrow{rm} t^*Saa \xRightarrow{rm} t^*aad$

Parse tree:-



5) Obtain LMP and RMP and parse tree for $(C(a,a), a, (aa))$ using

$S \rightarrow (L) \mid a$

$L \rightarrow L_1S \mid S$

6) Obtain LMP and RMP and parse tree for $O-(C(I^*O)-O)$ using

$E \rightarrow E-T \mid T$

$T \rightarrow T^*F \mid F$

$F \rightarrow (C) \mid O \mid I$

LMP:- $S \xRightarrow{lm} E-T$

$\xRightarrow{lm} T-T$

$\xRightarrow{lm} F-T$

$\xRightarrow{lm} O-T$

$\xRightarrow{lm} O-F$

$\xRightarrow{lm} O-(CE)$

$\xRightarrow{lm} O-(E-T)$

$\xRightarrow{lm} O-(T-T)$

$\xRightarrow{lm} O-(F-T)$

$\xRightarrow{lm} O-(CE-T)$

$\xRightarrow{lm} O-(T-T)$

$\xRightarrow{lm} O-(T^*F-T)$

$\xRightarrow{lm} O-(F^*F)-F$

$\xRightarrow{lm} O-(C(I^*O)-O)$

$\xRightarrow{lm} O-(C(I^*O)-O)$

RMP:- $S \xRightarrow{rm} E-T$

$\xRightarrow{rm} E-F$

$\xRightarrow{rm} E-(E)$

$\xRightarrow{rm} E-(E-T)$

$\xRightarrow{rm} E-(E-F)$

$\xRightarrow{rm} E-(E-O)$

$\xRightarrow{rm} E-(T-O)$

$\xRightarrow{rm} E-(F-O)$

$\xRightarrow{rm} E-(E-O)$

$\xRightarrow{rm} E-(T-T)$

$\xRightarrow{rm} E-(T^*O)-O$

$\xRightarrow{rm} E-(C(F^*O)-O)$

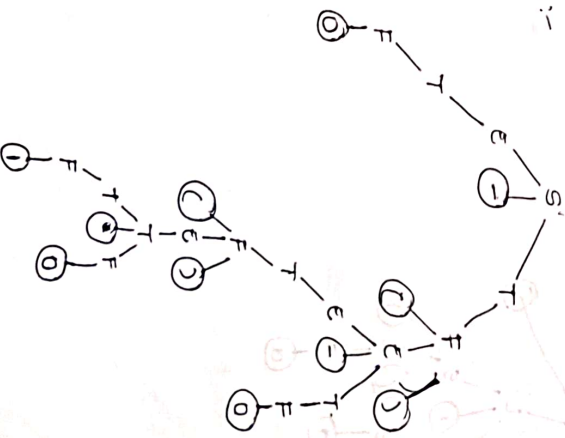
$\xRightarrow{rm} E-(C(I^*O)-O)$

$\xRightarrow{rm} T-(C(I^*O)-O)$

$\xRightarrow{rm} F-(C(I^*O)-O)$

$\xRightarrow{rm} O-(C(I^*O)-O)$

Parse tree:-



for the string

2m07

Parse Tree

2507

$$\begin{aligned} 5 &\rightarrow x \\ &\rightarrow dx \\ &\rightarrow aax \\ &\rightarrow aaox \\ &\rightarrow aaoaa \end{aligned}$$

Since 2 parse trees are different, grammar is ambiguous.

~~True~~ a) Check the ambiguity for the grammar

$$S \rightarrow aB / bA$$
$$A \rightarrow aS/bAA/a$$
 $B \rightarrow b\bar{s} / a\bar{c}B/b$

for the string $w = \text{"aabbab"}$

5. $\overline{LMP}$

Parade Tree

$S \rightarrow aB$
 $\rightarrow aAB$
 $\rightarrow aabbB$
 $\rightarrow aabbs$
 $\rightarrow aabbas$

→ aabbaa

LMDD

Pause tree 2

$$\begin{aligned} \emptyset &\rightarrow aB \\ . &\rightarrow aAB \\ &\rightarrow aaB \\ &\rightarrow aabBB \\ &\rightarrow aabbAB \\ &\rightarrow aabbaB \end{aligned}$$

Since a parse tree is different, grammar is ambiguous

d) Is the following grammar is ambiguous for the string "ibittaea"

5-3 ict5 / ict5es la

$$c \rightarrow b$$

2019

Parse Tree

- ic tses.
- ib t r c t s e s
- i b t i b t s e s
- i b t i b t a e s
- i b t i b t a e a

—

edw7

5 → i c t s

→ ibts

→ inhibitors

→ inhibits

→ ibibitapen

→ ibitaea

Since 2 parse trees are different, grammar is ambiguous

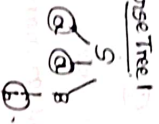
A hand-drawn diagram of a binary tree structure. The root node is 'S'. It has a left child 'l' and a right child 'c'. Node 'c' has a left child 'f'. Node 'f' has a left child 's'. Node 's' has a left child 'p' and a right child 'q'. Node 'q' has a left child 'e'. The tree is drawn with lines connecting the nodes, and the nodes are labeled with letters in circles.

5) Show that the following grammar is ambiguous for the string "aab".

$S \rightarrow AB|aAB$
 $A \rightarrow a|Ba$
 $B \rightarrow b$

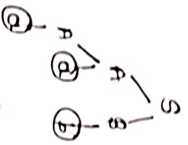
LMD

$S \rightarrow aAB$
 $\rightarrow aab$



LMD2

$S \rightarrow AB$
 $\rightarrow AaB$
 $\rightarrow aAB$
 $\rightarrow aab$



Since 2 parse trees are different, grammar is ambiguous.

6) $S \rightarrow aSbS|bSas|e$, string = "aabaabb"

7) Show that the balanced parenthesis grammar is ambiguous.

$S \rightarrow (S) | SS | e$

String = (()) ()

* Unambiguous Grammar: A grammar can be unambiguous

if the grammar doesn't contain ambiguity that means if it doesn't contain more than one leftmost derivation or more than one rightmost derivation or more than one parse tree for the given input string.

8) Obtain unambiguous Grammar for arithmetic expression grammar $E \rightarrow E+E | E-E | E^*E | E/E | id$.

The ambiguity from arithmetic expression can be removed by

Considering the precedence of the operator in the following order.

* Identifications - a, b, c (highest)

* Parenthesis ()

* Multiplication & division (* /)

* Addition & Subtraction (+, -)

* Associativity is considered from left to right

* The grammar can be written as.

* $I \rightarrow a|b|c$.

* $F \rightarrow (E) | I$.

* $T \rightarrow T^*F | T/F | F$

* $E \rightarrow E+T | E-T | T$.

* $E \rightarrow E^*T | E/T | T$.

* $T \rightarrow T^*F | T/F | F$

* $F \rightarrow (E) | I$

* $I \rightarrow a|b|c$.

Derive the string (a+b)*(a-c) using the above unambiguous grammar & write parse tree

$E \rightarrow T$

$\rightarrow T^*F$

$\rightarrow F^*F$

$\rightarrow (E)^*F$

$\rightarrow (E+T)^*F$

$\rightarrow (T+T)^*F$

$\rightarrow (I+T)^*F$

$\rightarrow (a+T)^*F$

parse tree

$$\frac{LMDI}{\%}$$

A diagram showing a hierarchical tree structure. The root node is 'p'. It has three children: 's', 'p', and 's'. The first 's' node has two children: 'p' and 's'. The second 'p' node has two children: 's' and 'p'. The third 's' node has two children: 'p' and 's'. The leaves are labeled with '3'.

Parse Tree 3

Parse Tree of

```

graph TD
    S1[S] --- S2[S]
    S1 --- P1[+]
    S1 --- S3[S]
    S2 --- LP1[(]
    S2 --- S4[S]
    S4 --- A[a]
    S4 --- P2[+]
    S4 --- S5[S]
    S5 --- LP2[(]
    S5 --- S6[S]
    S6 --- C[c]
    S6 --- P3[+]
    S6 --- S7[S]
    S7 --- M[*]
    S7 --- S8[S]
    S8 --- LP3[(]
    S8 --- S9[S]
    S9 --- D[d]
    S9 --- P4[+]
    S9 --- S10[S]
    S10 --- E[e]
  
```

11

2mD1

Parse Tree

```

graph TD
    S1[S] --- S2[S]
    S1 --- P[P]
    S2 --- LP1["("]
    S2 --- S3[S]
    S3 --- a[a]
    S3 --- plus1["+"]
    S3 --- S4[S]
    S4 --- LP2["("]
    S4 --- S5[S]
    S5 --- c[c]
    S5 --- plus2["+"]
    S5 --- S6[S]
    S6 --- d[d]
    S6 --- RP1[")"]
    S4 --- RP2[")"]
    P --- RP3[")"]
  
```

Parse Tree

一

① Write inherently ambiguous CFG by considering $L = \{a^n b^n c^m\}$.

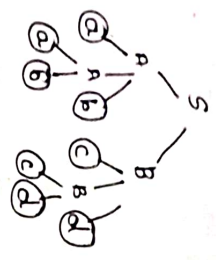
A grammar for L is

$$\begin{aligned} S &\rightarrow AB|C \\ A &\rightarrow aAb|ab \\ B &\rightarrow CBd|cd \\ C &\rightarrow aCd|apd \\ D &\rightarrow bDc|bc \end{aligned} \quad \xrightarrow{a^*cd} (a^*a^*c^*d^*)$$

Derivation-1
 $S \rightarrow AB$

$\rightarrow aAbB$
 $\rightarrow aabbb$
 $\rightarrow aabBCb$
 $\rightarrow aabbcdd$

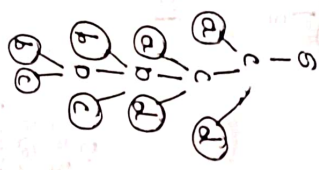
Parse Tree 1



Derivation-2
 $S \rightarrow c$

$\rightarrow aAd$
 $\rightarrow aadd$
 $\rightarrow aabDcd$
 $\rightarrow aabccdd$

Parse Tree 2



2) Write inherently ambiguous grammar for
 $L = \{a^n b^n c^m / n, m \geq 0\} \cup \{a^n b^m c^n / n, m \geq 0\}$.

A grammar for L is

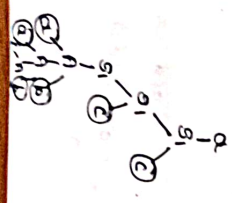
$S \rightarrow S_1 | S_2$
 $S_1 \rightarrow aS_1c | A$
 $A \rightarrow aAb | \epsilon$
 $S_2 \rightarrow aS_2b | B$
 $B \rightarrow bBc | \epsilon$

String = "aabbc"

Derivation 1

$S \rightarrow S_1$
 $\rightarrow aS_1c$
 $\rightarrow aS_1cS_1c$
 $\rightarrow aS_1c$
 $\rightarrow aAbc$
 $\rightarrow aabbc$

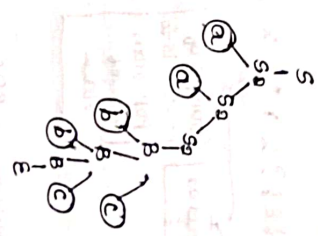
Parse Tree 1



Derivation 2

$S \rightarrow S_2$
 $\rightarrow aS_2$
 $\rightarrow aS_2$
 $\rightarrow aAb$
 $\rightarrow aabBc$
 $\rightarrow aabBc$
 $\rightarrow aabbc$

Parse tree 2



Push Down Automata (PDA) :- (what is PDA)

- * Pushdown automata is a way to implement a CFG.
- * A DFA can remember a finite amount of information, but a PDA can remember an infinite amount of information.
- * Push down automata is simply an NFA augmented with an "external state & memory".



Definition of a Pushdown Automata :-

* Formal definition notation for a PDA involves 7 components. We write the specification of a PDA P as follows:-

$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

Q is a finite set of states
 Σ is the input alphabet.
 Γ is the stack alphabet.

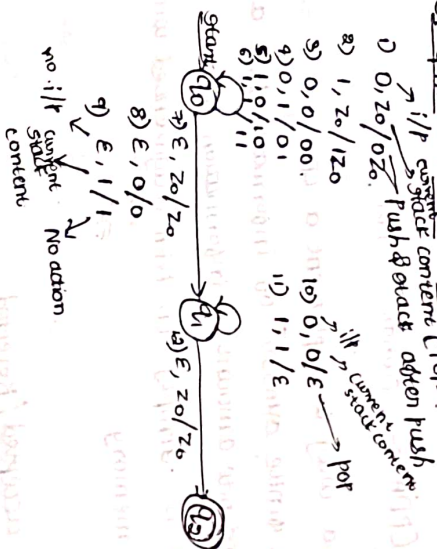
$q_0 \in Q$ is the initial state, $Z_0 \in \Gamma$ is bottom of stack.
 $Z_0 \in F \subseteq Q$ is the set of accepting states and

δ is the transition relation. It is a finite subset of

$$(Q \times (\Sigma \cup \{\epsilon\}) \times V^*) \times (Q \times V^*)$$



* Graphical notation of PDA :-



* Configuration & computations (Instantaneous Description or ID) :-

* A configuration of PDA P is an element of $Q \times \Sigma^* \times V^*$

* Acceptance by Final state :- let $P = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$ be a PDA.

Then, $L(P)$, the language accepted by P by final state, is

$$\{w \mid (q_0, w, z_0) \vdash^* (q, \epsilon, \alpha)\}$$

for some state q in F & any stack string α . That is, starting in the initial ID with w waiting on the input, P consumes w from the input & enters an accepting state. The contents of the stack at that time is irrelevant.

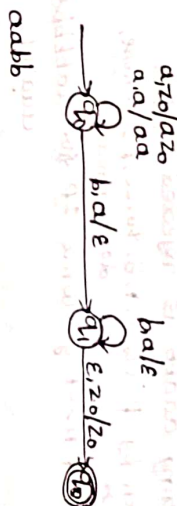
* Acceptance by Empty stack :- For each PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$,

we also define $NCP = \{w \mid (q_0, w, z_0) \vdash^* (q, \epsilon, \epsilon)\}$ for any state q . That is, NCP is the set of inputs w that P can consume & at the same time empty its stack.

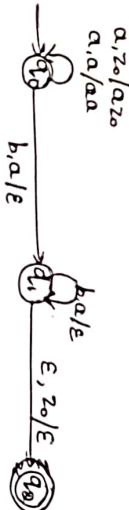
1) Construct a PDA for $L = \{a^n b^n \mid n \geq 1\}$. Write ID for $aabb$ & $babb$.

Final state method :-

Logic :- (If the input is a , push it on to the stack. If the input is b , pop it from the stack).



Empty stack method :-



Instantaneous Description :-

* $(q_0, aabb, z_0) \vdash (q_0, abb, az_0)$

$\vdash (q_0/bb, aaz_0)$

$\vdash (q_1/b, az_0)$

$\vdash (q_1/\epsilon, z_0)$

$\vdash (q_2/\epsilon, z_0)$

String is accepted.

* $(q_0, abb, z_0) \vdash (q_0, b, az_0)$

$\vdash (q_1, b, z_0)$

* No transition, string abb is rejected by PDA.

* aaabbb.

$(q_0, aaabbb, z_0) \vdash (q_0, aabb, az_0)$

$\vdash (q_0, abbb, aaaz_0)$

$\vdash (q_0, bbb, aaaaz_0)$

$\vdash (q_1, b, aaaaz_0)$

$\vdash (q_1, \epsilon, aaz_0)$. Input is empty but stack is not.

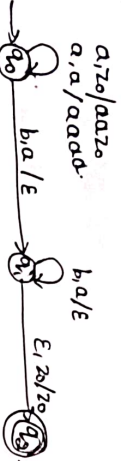
No transition, string aaabbb is rejected by PDA.

$\therefore$ PDA for $L = \{a^n b^n | n \geq 1\}$ is given by $P = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$ where,

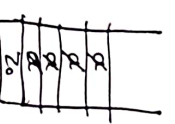
$Q = \{q_0, q_1, q_2\}$, $\Sigma = \{a, b\}$, $\Gamma = \{a\}$, δ is deterministic.

2) Construct a PDA for $L = \{a^n b^n | n \geq 1\}$. Write TD for aaabbb.

Final state method:-



aaabbb



* TD:- aaabbb

$(q_0, aaabbb, z_0) \vdash (q_0, aabb, az_0)$

$\vdash (q_0, bbb, aaaaz_0)$

$\vdash (q_1, bbb, aaaz_0)$

$\vdash (q_1, bb, aaaz_0)$

$\vdash (q_1, b, aaaz_0)$

$\vdash (q_1, \epsilon, az_0)$

$\vdash (q_2, \epsilon, az_0)$

string is accepted.

* TD:- aaabbb

$(q_0, aaabbb, z_0) \vdash (q_0, aabb, az_0)$

$\vdash (q_0, abbb, aaaaz_0)$

$\vdash (q_0, bbb, aaaz_0)$

$\vdash (q_1, bbb, aaaz_0)$

$\vdash (q_1, b, aaaz_0)$

$\vdash (q_1, \epsilon, aaaz_0)$

Input is empty but stack is not. String is rejected.

$\therefore$ PDA for $L = \{a^n b^n | n \geq 1\}$ is given by

$P = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$ where,

$Q = \{q_0, q_1, q_2\}$.

$\Sigma = \{a, b\}$

$\Gamma = \{a\}$

$\delta = \{ \delta(q_0, a, z_0) = (q_0, aaaz_0), \delta(q_0, a, a) = (q_0, aaa),$

$\delta(q_0, b, a) = (q_1, \epsilon), \delta(q_1, b, a) = (q_1, \epsilon), \delta(q_1, \epsilon, z_0) = (q_2, z_0) \}$.

$q_0 = \{q_0\}$

$z_0 = \{z_0\}$.

$F = \{q_2\}$.

3) Construct a PDA for $L = \{w c w^R | w \in \{a, b\}^*\}$. Write TD for babcbab.

Logic:- If input is a or b, push it onto the stack. If input is c, change the state. If input is a or b, pop it from the stack.

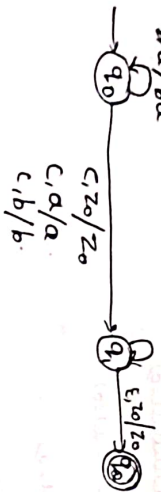
With respect to the following conditions:

i) If $P = a$, stack top = $a \Rightarrow$ pop.

ii) If $P = b$, stack top = $b \Rightarrow$ pop.

$a, z_0/a, z_0$
 $b, z_0/b, z_0$
 $a, a/a, a$
 $a, b/a, b$
 $b, b/b, b$
 $b, a/b, a$

$a, a/\epsilon$
 $b, b/\epsilon$



Formal Definition :- PPA is given by,

$P = \{Q, \epsilon, \delta, r, q_0, z_0, F\}$ where,

$Q = \{q_0, q_1, q_2\}$

$\epsilon = \{a, b, c\}$

$\delta = \{a, b\}$

$\delta = \{ \delta(q_0, a, z_0) = (q_0, a, z_0), \delta(q_0, b, z_0) = (q_0, b, z_0),$

$\delta(q_0, a, a) = (q_0, a, a), \delta(q_0, a, b) = (q_0, b, b),$

$\delta(q_0, b, a) = (q_0, b, a), \delta(q_0, c, z_0) = (q_1, z_0), \delta(q_0, c, a) = (q_1, a),$

$\delta(q_0, c, b) = (q_1, b), \delta(q_1, a, a) = (q_1, \epsilon), \delta(q_1, b, b) = (q_1, \epsilon),$

$\delta(q_1, \epsilon, z_0) = (q_2, z_0) \}$

$q_0 = \{q_0\}$

$z_0 = \{z_0\}$

$F = \{q_2\}$

* IP :- $babcbab$

$(q_0, babcbab, z_0) \vdash (q_0, abcbab, b, z_0)$

$\vdash (q_0, bcbab, ab, z_0)$

$\vdash (q_0, cbab, bab, z_0)$

$\vdash (q_1, bab, bab, z_0)$

$\vdash (q_1, ab, ab, z_0)$

$\vdash (q_1, b, b, z_0)$

$\vdash (q_1, \epsilon, z_0)$

$\vdash (q_2, \epsilon, z_0)$

String is accepted.

* IP :- $abcbab$

$(q_0, abcbab, z_0) \vdash (q_0, bcbab, a, z_0)$

$\vdash (q_0, cab, ba, z_0)$

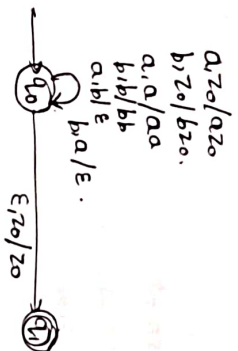
$\vdash (q_1, ab, ba, z_0)$

No transition from q_1 , string is Rejected.

It is deterministic.

4) Construct a PPA for $L = \{w \mid w \in \{a, b\}^* \text{ and } \#a(w) = \#b(w)\}$.

Write IP for $ababab$ and $abab$.



$\therefore$ PPA is given by,

$P = \{Q, \epsilon, \delta, r, q_0, z_0, F\}$ where,

$Q = \{q_0, q_1\}$

$\epsilon = \{a, b\}$

$r = \{a, b\}$

$\delta = \{ \delta(q_0, a, z_0) = (q_0, a, z_0),$

$\delta(q_0, b, z_0) = (q_0, b, z_0),$

$\delta(q_0, a, a) = (q_1, a, a),$

$\delta(q_0, b, b) = (q_1, b, b),$

$\delta(q_1, a, b) = (q_1, \epsilon),$

$\delta(q_1, b, a) = (q_1, \epsilon),$

$\delta(q_1, \epsilon, z_0) = (q_1, z_0) \}$

$q_0 = \{q_0\}$

$z_0 = \{z_0\}$

$F = \{q_1\}$

* IP :- $ababab$

$(q_0, ababab, z_0) \vdash (q_0, babab, a, z_0)$

$\vdash (q_0, abab, a, z_0)$

$\vdash (q_0, abb, a, z_0)$

$\vdash (q_0, abb, a, z_0)$

$\vdash (q_0, abb, a, z_0)$

$\vdash (q_0, bb, a, z_0)$

$\vdash (q_0, b, a, z_0)$

$\vdash (q_0, \epsilon, a, z_0)$

$\vdash (q_1, \epsilon, z_0)$

* IP :- $abab$

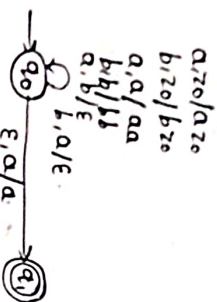
$(q_0, abab, z_0) \vdash (q_0, bab, a, z_0)$

$\vdash (q_0, bab, z_0)$

String is Rejected

5) Construct a PFA for $L = \{w \mid w \in \{a,b\}^* \text{ and } \#a(w) > \#b(w)\}$.

Write ID for a^2aa and $aabbb$.



$\therefore$ PFA is given by,

$P = \{q, \varepsilon, \delta, \gamma, q_0, z_0, F\}$ where,

$Q = \{q_0, q_1\}$.

$\Sigma = \{a, b\}$

$\gamma = \{q, b\}$

$\delta = \{ \delta(q_0, a, z_0) = (q_0, a, z_0),$

$\delta(q_0, b, z_0) = (q_0, b, z_0), \delta(q_0, a, a) = (q_0, aa),$

$\delta(q_0, b, b) = (q_0, bb), \delta(q_0, a, b) = (q_0, \varepsilon),$

$\delta(q_0, b, a) = (q_0, \varepsilon), \delta(q_0, \varepsilon, a) = (q_1, a) \}$.

$q_0, \{z_0\}, z_0 = \{z_0\}, F = \{q_1\}$.

ID:- a^2aa

$aabbb$

$(q_0, a^2aa, z_0) \vdash (q_0, a^2aa, a, z_0)$

$(q_0, aabbb, z_0) \vdash (q_0, aabbb, a, z_0)$

$\vdash (q_0, aa, z_0)$

$\vdash (q_0, bb, aa, z_0)$

$\vdash (q_0, a, a, z_0)$

$\vdash (q_0, b, aa, z_0)$

$\vdash (q_0, \varepsilon, aa, z_0)$

$\vdash (q_0, \varepsilon, z_0)$

No transition. $\therefore$ string

No Transition.

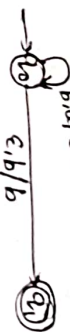
is rejected $\vdash (q_1, \varepsilon, aa, z_0)$

$\therefore$ string is rejected.

$\therefore$ string is accepted

6) $\#a(w) < \#b(w)$

a^2z_0/a^2z_0
 b^2z_0/b^2z_0
 $a, a/aa$
 $b, b/bb$
 $a, b/\varepsilon$
 $b, a/\varepsilon$



$\therefore$ PFA is given by, $P = \{q, \varepsilon, \gamma, \delta, q_0, z_0, F\}$ where,

$Q = \{q_0, q_1\}$.

$\Sigma = \{a, b\}$.

$\gamma = \{q, a, b\}$.

$\delta = \{ \delta(q_0, a, z_0) = (q_0, a, z_0), \delta(q_0, b, z_0) = (q_0, b, z_0),$

$\delta(q_0, a, a) = (q_0, aa), \delta(q_0, b, b) = (q_0, bb),$

$\delta(q_0, a, b) = (q_0, \varepsilon), \delta(q_0, b, a) = (q_0, \varepsilon),$

$\delta(q_0, \varepsilon, b) = (q_1, b) \}$.

$q_0 = \{q_0\}, z_0 = \{z_0\}, F = \{q_1\}$.

ID:- $aabbb$

a^2aa

$(q_0, aabbb, z_0) \vdash (q_0, aabbb, a, z_0)$

$(q_0, a^2aa, z_0) \vdash (q_0, a^2aa, a, z_0)$

$\vdash (q_0, bbb, aa, z_0)$

$\vdash (q_0, ab, b, z_0)$

$\vdash (q_0, bb, aa, z_0)$

$\vdash (q_0, b, aa, z_0)$

$\vdash (q_0, b, z_0)$

$\vdash (q_0, \varepsilon, z_0)$

$\vdash (q_0, \varepsilon, b, z_0)$

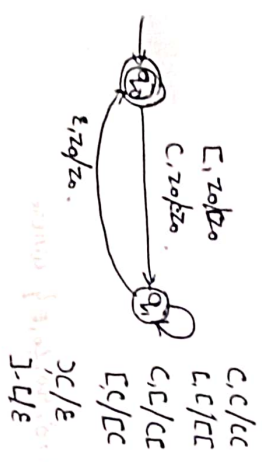
No Transition. $\therefore$ string is

$\vdash (q_1, \varepsilon, b, z_0)$

rejected.

$\therefore$ string is accepted

Construct a PDA for balanced parentheses having $(, (,),)$
 write ID for $() [()]$ and $[(())]$



PDA is given by,

$P = \{Q, \Sigma, \delta, \Gamma, q_0, Z_0, F\}$ where,

$Q = \{q_0, q_1\}$

$\Sigma = \{(, (,),)\}$

$\Gamma = \{c, c\}$

$\delta = \{(q_0, (, z_0) = (q_0, c, z_0), \delta(q_0, c, z_0) = (q_1, c, c), \delta(q_1, c, c) = (q_1, c, c), \delta(q_1, (, c) = (q_1, c, c), \delta(q_1,), c) = (q_0, c, c), \delta(q_1,), c) = (q_0, z_0, z_0)\}$

$q_0 = \{q_0\}$

$Z_0 = \{z_0\}$

$F = \{q_0\}$

* ID :- $() [()]$

$(q_0, () [()]) \vdash (q_1, () [()])$

$\vdash (q_0, [()])$

$\vdash (q_1, [()])$

$\vdash (q_0, [()])$

$\vdash (q_1, [()])$

$\vdash (q_0, [()])$

$\vdash (q_1, [()])$

String is accepted

* ID :- $[(())]$

$(q_0, [(())]) \vdash (q_1, [(())])$

$\vdash (q_1, [()])$

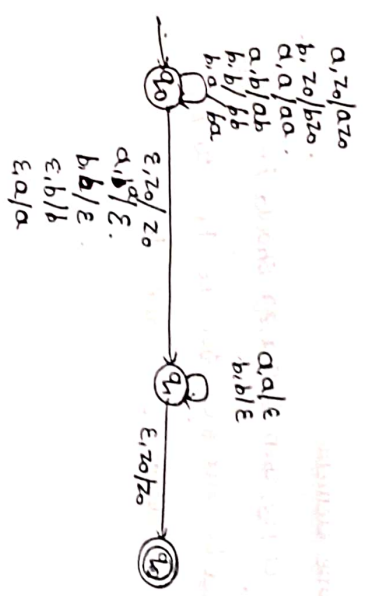
$\vdash (q_0, [()])$

$\vdash (q_1, [()])$

$\vdash (q_0, [()])$

String is rejected

8) Construct a PDA for $L = \{w^R \mid w \in \{a, b\}^*\}$ (even length palindromes). Write ID for $abbbba$ and $abaab$



It is highly non-deterministic PDA.

* ID :- $abbbba$.

$(q_0, abbbba, z_0) \vdash (q_1, abbbba, a, z_0)$

$\vdash (q_1, bbbba, ba, z_0)$

$\vdash (q_1, bba, bba, z_0)$

$\vdash (q_1, ba, bba, z_0)$

$\vdash (q_1, a, ba, z_0)$

$\vdash (q_1, e, z_0)$

$\vdash (q_1, e, z_0)$

$\vdash (q_1, e, z_0)$

* ID :- $abaab$.

$(q_0, abaab, z_0) \vdash (q_1, abaab, a, z_0)$

$\vdash (q_1, aab, ba, z_0)$

$\vdash (q_1, ab, ba, z_0)$

$\vdash (q_1, a, ba, z_0)$

$\vdash (q_1, e, z_0)$

$\vdash (q_1, e, z_0)$

String is rejected

String is accepted

PDA is given by, $P = \{Q, \Sigma, \delta, \Gamma, q_0, Z_0, F\}$ where, $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{a, b\}$, $\Gamma = \{a, b\}$, $\delta = \{(q_0, a, z_0) = (q_0, a, z_0), \delta(q_0, b, z_0) = (q_0, b, z_0), \delta(q_0, a, a) = (q_1, a, a), \delta(q_0, b, b) = (q_1, b, b), \delta(q_1, a, a) = (q_1, a, a), \delta(q_1, b, b) = (q_1, b, b), \delta(q_1, a, b) = (q_2, a, b), \delta(q_1, b, a) = (q_2, b, a), \delta(q_2, a, a) = (q_1, a, a), \delta(q_2, b, b) = (q_1, b, b), \delta(q_2, a, b) = (q_1, a, b), \delta(q_2, b, a) = (q_1, b, a), \delta(q_1, a, z_0) = (q_1, a, z_0), \delta(q_1, b, z_0) = (q_1, b, z_0)\}$

$q_0 = \{q_0\}$, $Z_0 = \{z_0\}$

$F = \{q_0\}$

String is accepted

String is rejected

String is accepted

String is accepted

INT Deterministic PDA:-

Defⁿ:- Let $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be a PDA. The PDA is deterministic if

1) $\delta(q, a, Z)$ has only one element.

2) If $\delta(q, \epsilon, Z)$ is not empty, then $\delta(q, a, Z)$ should be empty.

Both the conditions should be satisfied for the PDA to be deterministic. If one of the conditions fails, the PDA is non-deterministic.

* Equivalence between CFG and PDA (CFG $\rightarrow$ PDA):-

Step 1: Let q_0 be the start state and Z_0 be the initial symbol on the stack. Push start symbol S onto stack and change the state to q_1 . The transition is given by $\delta(q_0, \epsilon, Z_0) = (q_1, S, Z_0)$.

Step 2: For each production of the form $A \rightarrow a\alpha$, introduce the transition $\delta(q_1, a, A) = (q_1, \alpha)$.

Step 3: Finally, in state q_1 , without consuming any input, change the state to q_f , which is an accepting state.

i.e. $\delta(q_1, \epsilon, Z_0) = (q_f, Z_0)$.

1) For the CFG below, obtain PDA.

$S \rightarrow aABc$

$A \rightarrow aB/a$

$B \rightarrow bA/b$

$c \rightarrow \alpha$.

(Format $\rightarrow$ terminal followed by non-terminals)

Let q_0 be the initial state, q_f be the final state and Z_0 be the initial symbol on the stack.

Step 1: $\delta(q_0, \epsilon, Z_0) = (q_1, S, Z_0)$.

Step 2:

Production of CFG	Transition of PDA
$S \rightarrow aABc$	$\delta(q_1, a, S) = (q_1, aBC)$
$A \rightarrow aB$	$\delta(q_1, a, A) = (q_1, B)$
$A \rightarrow a$	$\delta(q_1, a, A) = (q_1, \epsilon)$
$B \rightarrow bA$	$\delta(q_1, b, B) = (q_1, aA)$
$B \rightarrow b$	$\delta(q_1, b, B) = (q_1, \epsilon)$
$c \rightarrow \alpha$	$\delta(q_1, a, c) = (q_f, \epsilon)$

Step 3: $\delta(q_1, \epsilon, Z_0) = (q_f, Z_0)$.



$a, s / nbc$
 $a, n / b$
 $a, n / e$
 $b, b / n$
 $b, b / e$
 $a, c / e$

2) $S \rightarrow a n b c / a n b$

$n \rightarrow a n b / a$

$b \rightarrow b n c / n$

$c \rightarrow c$

So CFG can be rewritten as,

$S \rightarrow a n b c / a n b$

$n \rightarrow a n b / a$

$b \rightarrow b n c / a n b / a$

$c \rightarrow c$

Let q_0 be the start state and z_0 be the initial symbol on the stack,
 q_f be the final state.

Step 1: $\delta(q_0, \epsilon, z_0) = (q_1, s, z_0)$

Step 2:

Production of CFG	Transition of PDA
$S \rightarrow a n b c$	$\delta(q_1, a, s) = (q_1, n b c)$
$S \rightarrow a n b$	$\delta(q_1, a, s) = (q_1, n b)$
$n \rightarrow a n b$	$\delta(q_1, a, n) = (q_1, n b)$
$n \rightarrow a$	$\delta(q_1, a, n) = (q_1, \epsilon)$
$b \rightarrow b n c$	$\delta(q_1, b, n) = (q_1, b c)$
$b \rightarrow a n b$	$\delta(q_1, b, n) = (q_1, n b)$
$b \rightarrow a$	$\delta(q_1, b, n) = (q_1, \epsilon)$
$c \rightarrow c$	$\delta(q_1, c, c) = (q_1, \epsilon)$

Step 3: $\delta(q_1, \epsilon, z_0) = (q_0, z_0)$



$a, s / nbc$
 $a, s / n b$
 $a, n / b$
 $b, b / n c$
 $a, b / n b$
 $a, b / e$
 $c, c / e$

3) $S \rightarrow a s b / a b$

[Hint: Rewrite CFG as $S \rightarrow a s b / a b$,
 $b \rightarrow b$]

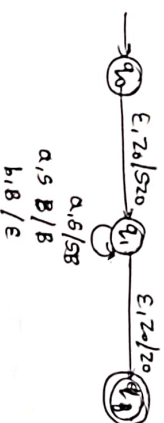
Let q_0 be the start state & z_0 be the initial symbol on the stack,
 q_f be the final state.

Step 1: $\delta(q_0, \epsilon, z_0) = (q_1, s, z_0)$

Step 2:

Production of CFG	Transition of PDA
$S \rightarrow a s b$	$\delta(q_1, a, s) = (q_1, s b)$
$S \rightarrow a b$	$\delta(q_1, a, s) = (q_1, b)$
$b \rightarrow b$	$\delta(q_1, b, b) = (q_1, \epsilon)$

Step 3: $\delta(q_1, \epsilon, z_0) = (q_0, z_0)$



4) Convert CFG to PDA.

$S \rightarrow a n a$

$n \rightarrow a s / b s / a$

5) $S \rightarrow a s b / a a b$

$[S \rightarrow a s b / a n b$

$b \rightarrow b$

$n \rightarrow a]$

4) $S \rightarrow aAB$
 $A \rightarrow aS/bS/a$

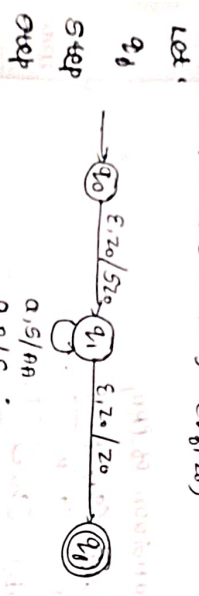
Let q_0 be the start state & Z_0 be the initial symbol on the stack, q_1 be the final state.

Step 1: $\delta(q_0, \epsilon, Z_0) = (q_1, S, Z_0)$

Step 2:

Production of CFG	Transition of PDA
$S \rightarrow aAB$	$\delta(q_1, a, S) = (q_1, AB)$
$A \rightarrow aS$	$\delta(q_1, a, A) = (q_1, S)$
$A \rightarrow bS$	$\delta(q_1, b, A) = (q_1, S)$
$A \rightarrow a$	$\delta(q_1, a, A) = (q_1, \epsilon)$

Step 3: $\delta(q_1, \epsilon, Z_0) = (q_1, Z_0)$



5) $S \rightarrow aSbh/aab$

CFG can be rewritten as,

$S \rightarrow aSBB/aAB$
 $B \rightarrow b$
 $A \rightarrow a$

Let q_0 be the start state & Z_0 be the initial symbol on the stack, q_1 be the final state.

Step 1: $\delta(q_0, \epsilon, Z_0) = (q_1, SZ_0)$

Step 2:

Production of CFG	Transition of PDA
$S \rightarrow aSBB$	$\delta(q_1, a, S) = (q_1, SBB)$
$S \rightarrow aAB$	$\delta(q_1, a, S) = (q_1, AB)$
$B \rightarrow b$	$\delta(q_1, b, B) = (q_1, \epsilon)$
$A \rightarrow a$	$\delta(q_1, a, A) = (q_1, \epsilon)$

Step 3: $\delta(q_1, \epsilon, Z_0) = (q_1, Z_0)$

